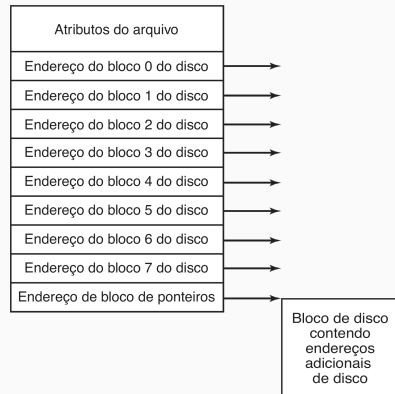


I-nodes

- outro método p/ monitorar quais blocos pertencem a quais arquivos é o *i-node*
 - *index-node* ou nó-índice
 - i.e., uma estrutura de dados listando os atributos e os endereços de disco dos blocos
- a partir do *i-node* de um arquivo podemos encontrar todos os blocos de disco do mesmo
- vantagem
 - o *i-node* precisa estar na memória apenas quando o arquivo estiver aberto
 - e.g., se cada *i-node* ocupa n bytes e um máximo de arquivos abertos é k , a memória total ocupada pelo array de *i-nodes* é de kn bytes



I-nodes

- o *array* de *i-nodes* é geralmente muito menor que o espaço ocupado pela tabela FAT
 - a tabela FAT precisa guardar (endereços de) todos os blocos
 - i.e., é proporcional ao tamanho do disco (cresce com o tamanho do disco)
 - já o esquema de *i-nodes* exige um espaço proporcional ao número máximo de arquivos abertos simultaneamente permitido
 - i.e., não importa se o disco tem 100 GB ou 10 000 GB
- desvantagem
 - o que fazer se o arquivo cresce além do limite fixo de endereços comportado pelo *i-node*?
 - solução é reservar o último endereço não para um bloco de dados mas para outro bloco contendo mais endereços

- antes de poder ler um arquivo é preciso abrir o mesmo
 - para abrir o SO usa o nome de caminho fornecido pelo usuário (e.g., `~/aulas/so/2026/aula34.pdf`) para encontrar a entrada de diretório no disco
 - por exemplo: `~/aulas/so/2026/`
 - a partir da entrada de diretório, é possível descobrir onde encontrar os blocos de disco relativos ao arquivo (`aula34.pdf`)
 - pode ser o endereço de disco do arquivo inteiro (alocação contígua)
 - ou o número do primeiro bloco (alocações encadeada e FAT)
 - ou o número do *i-node*
 - a função do sistema de diretórios é mapear o nome do arquivo em ASCII na informação necessária pra localizar os dados

- outro ponto a ser considerado são os atributos do arquivo
 - e.g., proprietário, momento de criação, etc)

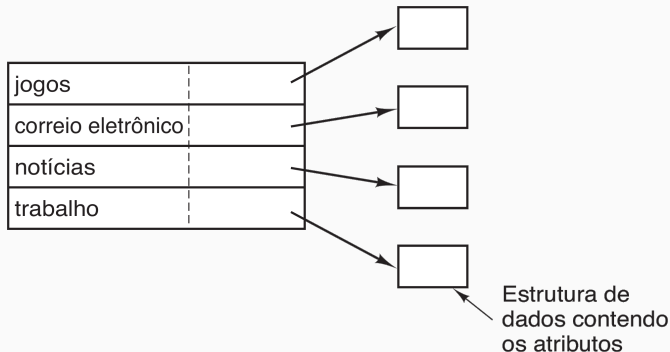
atributos armazenados diretamente na entrada de diretório

- e.g., cada diretório é composto de uma lista de entradas de tamanho fixo
 - uma entrada por arquivo
 - cada entrada contém
 - nome do arquivo (tamanho fixo)
 - estrutura dos atributos do arquivo
 - um ou mais endereços de disco (para os blocos)

jogos	atributos
correio eletrônico	atributos
notícias	atributos
trabalho	atributos

atributos armazenados nos *i-nodes*

- os atributos não ficam na entrada de diretório mas sim no próprio *i-node*
- a entrada de diretório pode ser apenas nome de arquivo e nº de *i-node*



IMPLEMENTANDO DIRETÓRIOS

v. FUR6, 2.3088.B, 2026 - Notas de Aula para Discentes (34)

- assumimos até aqui nomes de arquivos curtos e de tamanho fixo
 - no MS-DOS têm de 1-8 caracteres + extensão opcional com 1-3 caracteres
 - no UNIX v7, nomes (incluindo extensões) tinham de 1-14 caracteres
 - sistemas modernos permitem nomes maiores e de tamanho variável

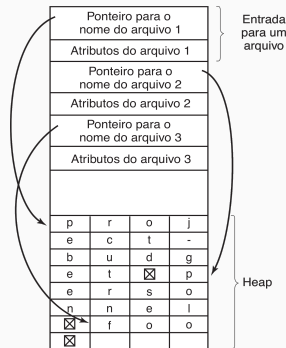
como implementar nomes maiores?

- alternativa simples é reservar um espaço maior pro nome (e.g., 255 caracteres)
 - desperdício → poucos nomes são longos
- opção melhor é assumir que o tamanho dos nomes é variável
 - i.e., cada entrada de diretório com 2 partes
 - uma porção fixa com o tamanho da entrada, proprietário, momento de criação, proteção, outros
 - outra porção com o nome do arquivo real (sem se importar com o tamanho)

Entrada para um arquivo	Tamanho da entrada do arquivo 1			
	Atributos do arquivo 1			
	p	r	o	j
	e	c	t	-
	b	u	d	g
	e	t	☒	
	Tamanho da entrada do arquivo 2			
	Atributos do arquivo 2			
	p	e	r	s
	o	n	n	e
	l	☒		
	Tamanho da entrada do arquivo 3			
	Atributos do arquivo 3			
	f	o	o	☒
	⋮			

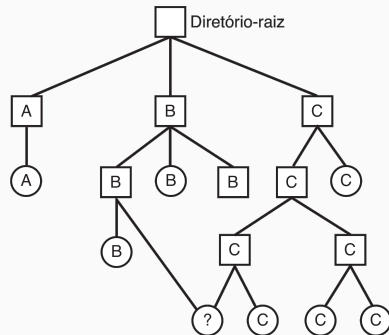
como implementar nomes maiores?

- a solução anterior tem a desvantagem de deixar lacunas de tamanho variável no diretório toda vez que um arquivo é removido
 - podemos usar compactação (já que a entrada de diretório está em memória)
- outra possibilidade é tornar fixo os tamanhos das entradas de diretórios e manter os nomes dos arquivos em um *heap* no fim de cada diretório



- quando nome de um arquivo precisa ser procurado, a opção trivial é fazer uma busca linear do início ao fim do diretório
 - pode ser lento se o diretório possui muitos arquivos
- podemos usar uma tabela *hash* de tamanho n para acelerar a busca
 - o nome do arquivo é mapeado para um valor entre 0 e $n - 1$
 - a entrada correspondendo ao código do *hash* é verificada
 - uma lista encadeada pode ser usada pra tratar colisões
 - desvantagem é tornar a administração do diretório mais complexas
 - pode ser uma opção viável em sistemas onde se espera que os diretórios frequentemente tenham milhares de arquivos
- outra técnica que pode acelerar a busca é o uso de cache
 - antes de começar a busca, verifica se o nome do arquivo está na cache
 - só funciona bem se a maioria das buscas for referente a um conjunto de pequeno de arquivos

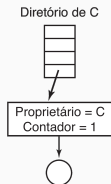
- muitas vezes é conveniente que um arquivo apareça simultaneamente em diretórios diferentes (até mesmo de usuários distintos)
 - evita duplicações
 - facilita trabalho em conjunto
- exemplo: um mesmo arquivo aparece simultaneamente em diretórios de usuários distintos (no caso B e C)
 - o arquivo é do usuário C
 - mas agora há um *link* para este arquivo em um diretório de B
- agora o sistema de arquivos não é mais tecnicamente uma árvore, mas sim um DAG (*Directed Acyclic Graph*)
 - complica um pouco a manutenção



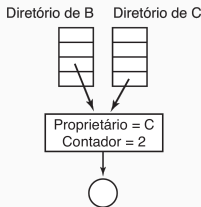
- compartilhamento adiciona alguns problemas
 - se os diretórios realmente contêm os endereços de (todos os) blocos, o *link* se efetivará numa cópia destes endereços
 - porém dessa maneira, quando B ou C adicionarem blocos ao arquivo, as adições não serão visíveis ao outro usuário!
 - uma solução é não listar os endereços diretamente no diretório mas sim em uma estrutura de dados associada ao arquivo (e.g., *i-node*)
 - outra solução é o sistema de arquivos criar um novo arquivo do tipo LINK e inserir este arquivo no diretório de B
 - o novo arquivo contém apenas o nome de caminho do alvo
 - quando B lê este arquivo, o sistema identifica que é do tipo LINK, verifica o nome e então faz a leitura
 - estratégia conhecida como ligação simbólica
- as duas soluções apresentam desvantagens

- na estratégia com *i-node* o problema reside na propriedade do arquivo
 - criar um *link* não altera o dono do arquivo, apenas aumenta o contador de *links*
 - o que acontece quando C decide remover o arquivo?
 - se remover o *i-node* teremos uma entrada de diretório em B apontando para um lugar inválido (pior, o *i-node* pode ser reaproveitado depois)
 - o sistema pode identificar (pelo contador) que o arquivo ainda está em uso (mas não dá pra saber facilmente por quem)
 - solução é remover a entrada de diretório em C mas deixar o *i-node* intacto
 - e se houver quotas de disco, o arquivo continuará impactando na cota de C!

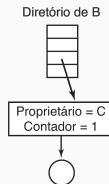
antes do link



após criar o link



C removeu o arquivo



- na estratégia com ligação simbólica não há este problema
 - já que apenas o verdadeiro proprietário tem um ponteiro pro *i-node*
 - as ligações guardam apenas o nome de caminho e não o endereço do *i-node*
 - quando o arquivo é removido por C, o *i-node* é liberado
 - qualquer tentativa de acessar o *link* vai falhar (impossível localizar o alvo)
 - remover um *link* simbólico não afeta o arquivo
- mas ligações simbólicas também têm problemas → sobrecarga
 - i.e., o arquivo com o nome de caminho precisa ser lido, analisado e então seguido → acessos adicionais ao disco
 - requer um *i-node* extra (+ 1 bloco de disco pro nome de caminho)
- *links* simbólicos tem a vantagem de não se limitarem a um sistema de arquivos ou à máquina local

- há um último aspecto que afeta ambos os tipos de ligações
 - agora podemos chegar aos arquivos por mais de um caminho!
 - um programa de *back-up* ou de transferência de dados poderá fazer múltiplas cópias
 - podemos ter *loops* também
 - ao restaurar a cópia em outro local teremos duas cópias distintas do arquivo (e não mais uma ligação para o mesmo arquivo)

- evolução tecnológica
 - CPUs mais rápidas
 - discos muito maiores e mais baratos
 - mas não muito mais rápidos
 - memórias crescem exponencialmente em tamanho
 - há um parâmetro que não tem se desenvolvido tão aceleradamente
 - tempo de busca dos discos
 - SSDs são exceção (comparativamente “não têm tempo de busca”)
- desempenho dos sistemas de arquivos se tornaram gargalos
 - motivação para novos sistemas de arquivos
 - e.g., **LFS** (*Log-structured File System*)

v. FUR6.23088.0.2020

- contexto
 - CPUs ficam mais rápidas
 - memórias RAM maiores
 - discos frequentemente incluem caches em disco
 - várias solicitações podem ser atendidas direto da cache
 - por consequência, a maior parte dos acessos será de escrita
- muitas operações de escrita são de pedaços pequenos
 - altamente ineficientes
 - e.g., uma escrita pequena leva $50 \mu s$ mas é precedida de uma busca de $10ms$ e um atraso de $4ms \rightarrow$ eficiência de $\approx 1\%$
- exemplo – escrita de um arquivo novo em UNIX
 - requer escrever i-node pro diretório, bloco do diretório, i-node pro arquivo e o arquivo em si
 - ideal é escrever imediatamente para evitar inconsistências em caso de queda

- LFS – sistema de arquivos estruturado em diário (*log*)
 - objetivo foi utilizar toda a largura de banda do disco
 - mesmo que a carga consista em pequenas operações de escrita
 - ideia é estruturar o disco inteiro como um grande diário (*log*)
 - periodicamente as operações de escrita no disco pendentes (i.e., que estão na memória) são agrupadas em um segmento e enviadas pro disco
 - tais dados são escritos logo depois do *log*
 - o segmento pode ter i-nodes, blocos de diretório, blocos de dados
 - no começo do segmento há um resumo do que ele contém
 - com um segmento médio de 1 MB já é possível usar bem a largura de banda
 - i-nodes continuam existindo (com os endereços dos blocos)
 - porém estão dispersos pelo *log* → encontrar um i-node é muito mais difícil
 - não dá pra encontrar só pelo número
 - solução: manter em disco (com cache em memória) uma tabela indexada pelo número do i-node apontando para o endereço deste no disco

- em resumo as escritas sempre ocorrem 1º na memória
 - em algum momento serão agrupadas e escritas em disco (*append* no final do diário)
- abertura de arquivo envolve encontrar a localização do i-node a partir do mapa em memória
 - após localizar o i-node é possível recuperar os endereços dos blocos
 - cada bloco estará em algum lugar do *log*
- quando parte de um arquivo é sobrescrita (ou ele é removido), o i-node é atualizado com os endereços dos novos dados
 - o que acontece com os dados antigos?
 - continuam ocupando espaço em segmentos escritos anteriormente!

- solução do LFS é usar uma thread limpadora
 - escaneia o *log* p/ compactá-lo
 1. lê o resumo de cada segmento p/ verificar quais i-nodes e arquivos estão ali
 2. confere o mapa atual de i-nodes p/ verificar se o i-node ainda é referenciado e se os blocos de arquivo ainda estão em uso
 3. caso não estejam, são descartados
 4. o que estiver em uso, vai para a memória para que seja reescrito em um novo segmento
 5. o segmento original lido é marcado como disponível
 - o disco vira um *buffer* circular
 - *thread* de escrita insere novos segmentos no início
 - *thread* limpadora remove do final
 - quando bloco é copiado p/ novo segmento, é preciso localizar o i-node, atualizá-lo, colocá-lo na memória p/ reescrita e atualizar o mapa de i-nodes p/ nova cópia
 - ainda assim o desempenho é comparável para leituras e escritas grandes
 - e supera por uma ordem de magnitude em escritas pequenas